FINQUEST

Hackathon Build Manual

4 People · 36 Hours · Win the Room

React Three Fiber · Langflow · Claude API · Firebase · Vercel

SECTION 01

Executive Summary

What you are building, what you will demo, and how to win

FinQuest is a web-based game that turns real-world spending habits into a living 3D city. Players categorise purchases — Need, Want, Treat, or Investment — and watch an isometric city react in real time. Good proportional allocation builds a glowing skyline. Poor habits bring storms and urban decay. The game teaches financial literacy through consequence, not instruction.

This manual tells 4 people exactly what to build, in what order, across 36 hours. It is optimised for one thing: a demo that wins. Not a production platform. Not a scalable architecture. A flawless, beautiful, technically impressive demo that tells a complete story in under 3 minutes.

The Golden Rule

The demo is the pitch. Every decision in this manual — what to build, what to fake, what to cut — is evaluated against one question: does this make the 3-minute demo loop work flawlessly and look incredible? If it does not serve the demo, it does not get built.

What	Decision
The Wow Moment	Apple Pay purchase → iOS Shortcut fires → notification → one tap → 3D city reacts instantly
AI Differentiator	Langflow RAG pipeline: Claude reads spending patterns, outputs structured city-change JSON
Multiplayer Hook	Two cities visible side by side, shared AI-triggered world event affects both simultaneously

What	Decision
Privacy Angle	Zero dollar amounts stored — proportion-only model, income-fair, privacy-safe by design
Tech Story	Five integrated systems: React Three Fiber city, Langflow AI pipeline, Firebase real-time, PWA push, payment trigger
Fallback Plan	Simulate Payment button triggers identical pipeline if Shortcut fails live — judges never notice

SECTION 02

Hackathon Tech Stack

Fast, stable, impressive — every layer decided

Frontend

Layer	Choice	Why for Hackathon	Skip Instead
Language	TypeScript	Catches prop errors in 3D data types — saves debugging time	JavaScript — type errors in Three.js hooks cost hours
Framework	React 18 + Vite	Fastest HMR, R3F requires React, zero config to start	Next.js — SSR adds complexity you don't need
State	Zustand	No boilerplate, works perfectly inside useFrame hooks	Redux — too verbose; Context — re-render issues with 3D
Routing	React Router v6	Simple, battle-tested, deep link support	TanStack Router — overkill
Styling	Tailwind CSS	Utility classes, no CSS files to manage, fast iteration	CSS Modules — too slow for hackathon speed
3D	React Three Fiber + Drei	React-native Three.js, hooks-based, huge example library	Plain Three.js — imperative style fights React state model

Backend

Layer	Choice	Why for Hackathon	Production Alternative
Runtime	Vercel Serverless Functions	Zero config, auto-deploys from GitHub, generous free tier	Express on Railway — more control but more setup
Language	TypeScript	Same language as frontend — no context switching	Python — better AI libs but splits the codebase mentally
Push Notifications	web-push npm library	VAPID standard, 50-line implementation	Firebase Cloud Messaging — simpler but needs native app for iOS

Database

Store	Technology	What Goes Here	Why Not Alternative
Real-time city state	Firebase Realtime DB	cityState, weather, live proportions, room events	Supabase Realtime — works but less proven under hackathon speed
Persistent records	Firestore	Profiles, transaction history, worlds, cities, streaks	PostgreSQL — requires a server you don't have time to configure
Client cache	Zustand (memory)	Current week data, UI state, Three.js inputs	LocalStorage — persists but adds sync complexity

AI & Orchestration

Layer	Choice	Why for Hackathon	When to Skip It
Orchestration	Langflow on Railway (Docker)	Visual pipeline editor — AI engineer can iterate without touching app code	Skip if Railway deploy takes >45min; fall back to direct Claude calls
Model	Claude API (claude-sonnet-4-20250514)	Best structured JSON output, reliable instruction-following	GPT-4o — comparable but different pricing; either works
RAG / Knowledge Base	Langflow built-in Chroma	Zero extra setup, financial literacy chunks load in minutes	Pinecone — better at scale, unnecessary for demo

Infrastructure

Service	Platform	Cost	Deploy Method
Frontend + API functions	Vercel	Free	git push — auto deploys in ~60 seconds
Langflow	Railway (Docker)	~$5	Deploy langflowai/langflow:latest image

Service	Platform	Cost	Deploy Method
Auth + Database	Firebase	Free (Spark plan)	Firebase console + SDK
3D Assets (if any)	Public GitHub repo or Vercel /public	Free	Committed to repo
Monitoring	Vercel built-in logs	Free	No setup needed

SECTION 03

Hackathon Architecture

How everything connects — optimised for demo reliability

Complete Data Flow

Every technical decision traces back to this pipeline. If this loop works flawlessly, you win.

1. User makes Apple Pay purchase on their phone
2. iOS Shortcut fires — POSTs { merchant, timestamp } to /api/webhook on Vercel
3. Webhook writes pendingTransaction to Firebase Realtime DB + sends Web Push notification
4. User taps notification — PWA opens to /log?merchant=Woolworths (pre-filled)
5. User taps one category button (Need / Want / Treat / Investment) — one tap, done
6. Client-side Proportion Engine recalculates weekly % allocation — instant, no network
7. Zustand state updates — Three.js city begins animating immediately (zero latency)
8. Transaction written to Firestore; cityState synced to Firebase Realtime DB
9. Vercel function calls Langflow /api/run with proportion payload
10. Langflow: RAG retrieval → prompt assembly → Claude → structured JSON output
11. JSON { weather_effect, building_changes, advisor_message } written to Firebase
12. All clients reading that cityId see the update — Three.js city fully reacts

What Can Be Faked Without Judges Noticing

The Langflow RAG knowledge base can be 10 hand-written text chunks instead of a proper corpus. Firebase Security Rules can be open during the demo. The friend city can load from hardcoded mock data. None of this is visible to judges — only the output matters.

Where Logic Lives — Hackathon Version

Logic	Lives In	Why Here, Not Elsewhere
Proportion % calculation	TypeScript client module	Pure math — zero latency, no network, testable in 10 minutes
City state mapping (% → 3D params)	TypeScript client module	Deterministic rules — must be instant for demo impact
AI advisor response	Langflow Advisor Flow	The one AI flow worth building — highest wow per engineering hour
World event (multiplayer)	Langflow World Event Flow or hardcoded JSON	If time allows; otherwise hardcode 3 impressive events to trigger manually
Push notification send	Vercel serverless function	Server-side only — needs VAPID private key
Webhook receive	Vercel serverless function	Public endpoint — must be on a real URL, not localhost

Logic	Lives In	Why Here, Not Elsewhere
Real-time city sync	Firebase Realtime DB listeners	Purpose-built for this — zero custom code for the sync itself

SECTION 04

Feature Priority List

Must-have, should-have, nice-to-have, and cut now

Must-Have — Do Not Demo Without These

Feature	Demo Value	Risk	Est. Hours	Owner
Transaction logging UI (4 buttons)	Critical	Low	2h	P1
Proportion engine (TypeScript)	Critical	Low	2h	P4
Three.js isometric city reacts to input	Critical	Medium	5h	P2
Weather system (clear/rain/storm)	Critical	Low	2h	P2
Claude API advisor response	Critical	Low	2h	P3
iOS Shortcut → webhook → push notification	Critical	Medium	4h	P3+P4
PWA installable on real phone	Critical	Low	1h	P1
Firebase Auth + user bootstrap	Critical	Low	2h	P4
Vercel production deployment	Critical	Low	1h	P4
Simulate Payment fallback button	Critical	Low	30min	P1

Should-Have — Build If Core Loop Done by Hour 14

Feature	Demo Value	Risk	Est. Hours	Owner
Friend city visible in 3D world	High	Medium	3h	P2
Shared world economic event	High	Medium	3h	P3+P4
Firebase real-time sync (multiplayer)	High	Low	3h	P4
Langflow RAG (vs direct Claude call)	High	Medium	3h	P3
Streak indicator on city	Medium	Low	2h	P1+P2
Android Automate trigger	Medium	Low	2h	P3

Nice-to-Have — Only If Everything Else is Stable

Feature	Demo Value	Risk	Est. Hours
Building upgrade visual tiers	Medium	Low	3h
Weekly settle animation	High	Medium	4h
Pattern analyser (weekly Langflow flow)	Medium	Medium	3h
Challenge system	Medium	Medium	4h
City share screenshot	Low	Low	2h

Cut These Now — Do Not Attempt

Feature	Why Cut
Real bank data via Plaid/Basiq	OAuth complexity alone kills 6+ hours — not worth it for the demo
Android Automate (if iOS Shortcut works)	Doubles QA surface — demo on iPhone only
GLTF 3D model imports	Asset pipeline complexity — procedural geometry looks just as good
Native iOS/Android app	PWA is sufficient and installable
Public world discovery	Moderation risk, scope explosion
Full Firebase Security Rules	Overkill for demo — leave open, add post-hackathon
Unit testing beyond proportion engine	Manual testing is sufficient in 36 hours

SECTION 05

Team Split for 4 People

Roles, ownership, and how to stay unblocked

Assign roles before the clock starts. No role switching. No asking someone else to do your job. Every hour spent context-switching is an hour not building.

P1 Frontend + UI + Demo Flow

PRIMARY OWNS:

- React app structure, routing, all screens
- Transaction logging UI — the 4-button screen
- PWA manifest + service worker config
- Zustand store setup (shared with P4)
- Mobile-responsive layout
- Simulate Payment fallback button
- Loading states, error states, transitions
- Demo flow polish — every screen judges see

BUILDS IN PARALLEL WITH:

P2 on Three.js scene integration into React canvas

P2 3D Engineer — Three.js World

PRIMARY OWNS:

- React Three Fiber canvas inside React app
- Isometric city: buildings, roads, ground
- Building height/count driven by Zustand state
- Weather system: rain particles, skybox, fog
- Citizens: low-poly figures, density from health
- Friend city: smaller city in the distance
- useFrame animation loops and lerp transitions
- Visual polish: lighting, shadows, camera orbit

BUILDS IN PARALLEL WITH:

P1 on how canvas embeds in page layout

P3 AI + Backend + Langflow

PRIMARY OWNS:

- Langflow Railway deployment
- Advisor Flow: RAG + Claude + JSON output
- World Event Flow (if time allows)
- Vercel serverless functions: /api/webhook, /api/advisor, /api/world-event
- /api/webhook: receives Shortcut POST, writes Firebase, sends push
- Web Push: VAPID keys, sendNotification function
- iOS Shortcut build and test
- Android Automate (stretch only)

BUILDS IN PARALLEL WITH:

P4 on Firebase schema and what data shape /api/webhook should write

P4 Firebase + Integration + DevOps

PRIMARY OWNS:

- Firebase project setup: Auth, Realtime DB, Firestore
- Zustand store definitions (shared contract with P1)
- Proportion engine TypeScript module
- City/World data schema design
- Firebase Realtime DB listeners → Zustand sync
- Vercel project setup, env vars, deployment
- GitHub repo, branching, merge gatekeeper
- Integration: connecting P1+P2+P3 modules together
- Demo device setup and pre-flight testing

BUILDS IN PARALLEL WITH:

Everyone — P4 is the integration owner and must stay unblocked

Coordination Rules

Rule	Why It Matters
P4 is the only person who merges to main	Prevents the merge conflict crisis that kills hackathon teams in hour 30
Everyone works on feature branches	P1/branch-ui, P2/branch-3d, P3/branch-ai, P4/branch-firebase
15-minute sync every 4 hours	Check progress, surface blockers, re-prioritise if behind
Shared mock data file from hour 1	P2 and P3 should never wait for real data — mock it and swap later
Feature freeze at hour 26	No new features after this point. Bug fixes only.
Demo device locked at hour 28	Do not change hardware or browser after this point

SECTION 06

36-Hour Execution Plan

Hour by hour — what gets built, by whom, by when

Hour-by-Hour Timeline

H0–1
Setup Sprint

- ALL: 30-min planning session — confirm roles, install tools, agree on data contracts
- P4: Create GitHub repo, Vite+React+TS scaffold, push to main, connect Vercel
- P3: Create Firebase project, enable Auth + Realtime DB + Firestore, share credentials
- P3: Create Railway account, deploy Langflow Docker image (langflowai/langflow:latest)
- P1: Clone repo, confirm dev server runs, set up Tailwind, create route skeleton
- P2: Confirm Three.js/R3F installs, create CityScene.tsx with a box rendering
- DELIVERABLE: Everyone has a running dev server. Vercel auto-deploys on push. Firebase live.

H1–3
Foundation Sprint

- P1: Build transaction logging screen — 4 category buttons, merchant field, submit handler
- P1: Build onboarding screen — sign in with Google, first city creation
- P2: Build ground plane, 5 placeholder BoxGeometry buildings in isometric scene
- P2: Wire OrbitControls, ambient + directional lighting, camera angle set
- P3: Write proportionEngine.ts — pure functions, add transaction, calc percentages, health score
- P3: Build Zustand gameStore with proportions, cityState, weekHistory
- P4: Firebase Auth flow — Google sign-in, session persistence, protected routes
- P4: On first sign-in: create /cities/{cityId} and /worlds/{worldId} in Firestore
- DELIVERABLE: Sign in works. Tap a button. Proportion updates in Zustand. 3D scene visible.

H3–6
Core Loop Sprint

- P1: Connect category buttons to proportionEngine — submit updates Zustand state
- P1: Build /log?merchant=X deep link handler — pre-fills merchant field from URL params
- P2: Build cityMapper.ts — map savingsPercent→bankHeight, wantsPercent→restaurantCount, health→weather
- P2: Wire cityMapper outputs to building heights via useFrame lerp — city animates on state change
- P3: Deploy Langflow Advisor Flow — Input→Chroma RAG→Prompt→Claude→JSON Output
- P3: Load 10 financial literacy text chunks into Chroma (50/30/20, emergency fund, etc.)
- P4: Build /api/advisor Vercel function — receives proportions, calls Langflow, returns JSON
- P4: Connect: after transaction submit → call /api/advisor → write advisor_message to Zustand
- DELIVERABLE: Tap a button → proportion changes → building height changes → advisor message appears

H6–10

- P3: Generate VAPID keys — store in Vercel env vars

Payment Trigger Sprint	• P3: Write /api/webhook Vercel function — receive POST, validate secret header, write to Firebase • P3: Write sendPushNotification function using web-push library • P3: Build iOS Shortcut — URL action POSTing {merchant, secret} to /api/webhook on production URL • P1: Register service worker in React app, call pushManager.subscribe(), store subscription in Firestore • P1: Service worker push event handler — showNotification with deep link URL • P4: Connect webhook→Firebase pendingTransaction→push notification chain end to end • ALL: Test full chain on a real iPhone — Shortcut fires → notification appears → tap → /log opens • P2: Add WeatherSystem.tsx — colour-shifting skybox based on cityState.weather enum • DELIVERABLE: iOS Shortcut fires. Push notification arrives. App opens pre-filled. City reacts.
H10–14 *Multiplayer + AI Polish Sprint*	• P4: Build Firebase Realtime DB bidirectional sync — local Zustand writes to DB, DB changes update Zustand • P4: Build World/Room join flow — 6-character room code, join screen, store worldId on user • P2: Build FriendCity.tsx — smaller city rendered at (x+50, 0, 0) using room member cityState from Firebase • P3: Build /api/world-event function — calls Langflow World Event Flow with all member proportions • P3: World event JSON written to /worlds/{worldId}/activeEvent — all clients receive and city weather shifts • P1: Build advisor chat panel as slide-up drawer — displays advisor_message with typing animation • P1: Build streak indicator — simple flame icon above city hall, count from Firestore • DELIVERABLE: Two devices in same room. Both cities visible. World event affects both simultaneously.
H14–18 *Polish Sprint*	• P2: Lighting polish — soften shadows, add warm golden-hour state for healthy cities • P2: Add rain particle system (small white spheres falling, opacity based on storm intensity) • P2: Add low-poly citizen capsules walking on roads — count tied to health score • P1: Mobile-responsive layout — test on real iPhone, fix touch targets, spacing • P1: Loading states on all async operations — spinner while Langflow responds • P1: Error states — if advisor fails, show cached message rather than broken UI • P3: Pre-cache 5 advisor responses for common demo scenarios as JSON fallback file • P4: PWA manifest.json — display: standalone, icons, theme_color, short_name • DELIVERABLE: App installable on iPhone. City looks beautiful. All screens polished.
H18–22 *Sleep / Buffer*	• MANDATORY: Everyone sleeps minimum 3–4 hours • P4 stays up 1 extra hour to document all known bugs and remaining tasks in a shared doc • Set an alarm. Bad decisions at hour 28 on zero sleep cause more failures than bad code. • Wake up with a clear list — no re-orientation time wasted
H22–28 *Integration + Demo Prep*	• ALL: Full demo run-through on real devices — identify every failure point • P4: Fix all critical bugs found in demo run — crashing, blank screens, broken sync • P3: Test Langflow under repeated rapid calls — confirm it handles demo pace without cold start lag

	• P3: Pre-warm Langflow by calling a health endpoint before demo starts — eliminate cold start • P2: Final Three.js polish — ensure 60fps on demo laptop, reduce draw calls if needed • P1: Build the Simulate Payment button — triggers identical webhook→push→log pipeline manually • P1: Confirm deep link works from notification tap on real iPhone • ALL: Demo rehearsal × 3 — one person speaks, one operates phone, one operates laptop • DELIVERABLE: Demo works perfectly 3 times in a row. Fallback button works. Script rehearsed.
H28–33 *Final Polish + Pitch*	• FEATURE FREEZE — No new code. Bug fixes only. • P1: Final UI sweep — typography, spacing, colour consistency across all screens • P2: Ensure city looks stunning as the first thing judges see — golden hour lighting default • P3: Write one-page architecture diagram — payment trigger → webhook → Firebase → Langflow → Three.js • ALL: Write pitch narrative — problem, solution, demo, tech story, vision (3 minutes total) • Anticipate 4 judge questions: retention, CommBank comparison, Apple Wallet reality, business model • Record 60-second backup demo video in case of live failure
H33–36 *Submission + Demo*	• H33: Final Vercel deployment. Confirm production URL is live and stable. • H33: Confirm Langflow Railway instance is warm — make one test call • H34: Submit project to hackathon platform with description, tech stack, team names • H34: Final demo rehearsal — exact words, exact taps, exact sequence • H35: Demo devices charged, notifications enabled, WiFi confirmed, Shortcut tested • H36: Demo. One speaker. One phone operator. One laptop operator. 3 minutes. Win.

SECTION 07

Langflow Integration Guide

What to build, how to connect it, and how to keep it reliable

What Langflow Does in FinQuest

Build two Langflow flows. Advisor Response Flow is essential and must be working by hour 6. World Event Flow is a should-have that can be attempted from hour 10 if the core loop is stable.

Flow 1 — Advisor Response Flow (MUST BUILD)

- Input Node: receives { needs_pct, wants_pct, treats_pct, investments_pct, recent_category }
- Chroma RAG Node: query knowledge base with recent_category as search term
- Load 10 text chunks into Chroma: 50/30/20 rule, emergency fund basics, savings habits, debt basics, needs vs wants definition, compound interest, spending triggers, weekly reset tips, proportional thinking, investment basics
- Prompt Node: combine retrieved context + player proportions → construct advisor prompt
- Claude Node: model=claude-sonnet-4-20250514, temperature=0.7, max_tokens=300
- JSON Output Node: format as { advisor_message: string, weather_effect: string, building_changes: object, tone: string }
- Export flow ID → store in LANGFLOW_ADVISOR_FLOW_ID env var

Flow 2 — World Event Flow (SHOULD BUILD)

- Input Node: receives array of all room members' cityStates
- Claude Node: given all member proportions, generate a shared economic event
- JSON Output Node: { event_type, description, weather_effect, duration_hours }

Biggest Wow Per Engineering Hour

The Advisor Flow with RAG is your highest-value Langflow investment. It takes 2-3 hours to build and produces responses that feel deeply personalised. Direct Claude API calls feel generic. Langflow with RAG feels like a real financial AI system — which is exactly what judges reward under Technical Difficulty.

Railway Deployment — Step by Step

13. Go to railway.app, create account, click New Project
14. Select Deploy from Docker Image, enter: langflowai/langflow:latest
15. Add environment variables: LANGFLOW_SECRET_KEY=your-random-secret, PORT=7860
16. Railway assigns URL: your-project.railway.app — copy this
17. Open Langflow UI at that URL, create API key in Settings
18. Store in Vercel env: LANGFLOW_URL=https://your-project.railway.app, LANGFLOW_API_KEY=your-key
19. Build your flows in the Langflow visual editor — export each as JSON to /langflow/flows/ in repo

Connecting to Vercel Functions

Your /api/advisor Vercel function is a thin proxy. The frontend never sees Langflow:

```
// /api/advisor.ts
export default async function handler(req, res) {
  const { proportions } = req.body;
  const response = await
fetch(`${process.env.LANGFLOW_URL}/api/v1/run/${process.env.LANGFLOW_ADVISOR_FLOW_ID}`,
{
    method: 'POST',
    headers: { 'x-api-key': process.env.LANGFLOW_API_KEY, 'Content-Type':
'application/json' },
    body: JSON.stringify({ input_value: JSON.stringify(proportions), input_type: 'text'
})
  });
  const data = await response.json();
  res.json(data.outputs[0].outputs[0].results.message.text);
}
```

Demo Reliability — Critical Rules

Risk	Mitigation
Langflow cold start (5-10s delay)	Pre-warm by calling /api/health 10 minutes before demo — Railway keeps instance warm
Langflow returns malformed JSON	Wrap parse in try/catch — on failure, serve advisor_fallback.json with 5 pre-written responses
Langflow Railway instance down	Have LANGFLOW_BYPASS=true env var — when set, /api/advisor skips Langflow and returns cached response
Claude API rate limit	Use exponential backoff in Vercel function — retry up to 3 times before fallback
Flow returns wrong JSON shape	Add a JSON schema validator step in the Vercel function — if invalid, use fallback

What Should NOT Be in Langflow

Do NOT use Langflow for	Use instead
Proportion calculation	TypeScript proportionEngine.ts — pure math, no latency acceptable here
Firebase reads/writes	Firebase SDK directly — Langflow DB connectors add unnecessary complexity
Web Push sending	web-push library in Vercel function — simple, reliable, no orchestration needed
Auth token validation	Firebase Admin SDK — security-critical path, not AI's job

Do NOT use Langflow for	Use instead
Three.js state updates	Zustand + React — UI concern entirely

SECTION 08

Three.js World Guide

Scene setup, assets, data connection, and performance

React Three Fiber vs Plain Three.js

Use React Three Fiber (R3F) without question. In a hackathon, the critical advantage is that your city buildings are React components — they re-render when Zustand state changes using the same mental model as your UI code. Plain Three.js requires imperative manipulation that fights React's rendering model and will cost P2 hours of debugging state synchronisation that R3F handles automatically via useFrame.

Install Everything Upfront

```
npm install three @react-three/fiber @react-three/drei @react-three/postprocessing
```

Folder Structure — Keep It Simple

```
src/three/
  CityScene.tsx          ← root Canvas, owns camera + lighting
  components/
    Ground.tsx            ← flat plane, road grid lines
    BuildingDistrict.tsx  ← InstancedMesh cluster per category
    WeatherSystem.tsx     ← skybox colour, rain particles, fog
    Citizens.tsx          ← capsule NPCs, density from health score
    FriendCity.tsx        ← second city from room member data
  hooks/
    useCityState.ts       ← reads Zustand, returns mapped 3D params
  utils/
    cityMapper.ts         ← pure functions: proportion% → 3D values
```

The cityMapper Pattern — Core of Everything

Every 3D visual is driven by this pure TypeScript mapping layer. Write it first. Test it manually. Everything else depends on it.

```
// cityMapper.ts — pure functions, no side effects
export const mapSavingsToBank = (pct: number) => 1 + (pct / 30) * 7;
export const mapWantsToRestaurants = (pct: number) => Math.max(1, Math.floor(pct / 8));
export const mapNeedsToApartments = (pct: number) => Math.max(2, Math.floor(pct / 10));
export const mapInvestmentsToTower = (pct: number) => 0.5 + (pct / 20) * 4;
export const mapHealthToWeather = (score: number): 'clear'|'overcast'|'rain'|'storm' =>
{
  if (score > 75) return 'clear';
  if (score > 50) return 'overcast';
  if (score > 30) return 'rain';
```

```
    return 'storm';
  };
```

Building Style — Procedural, Not GLTF

Do not import 3D model files. Use Three.js geometry primitives assembled in code. This is faster to build, loads instantly, looks intentionally designed, and has zero asset pipeline to manage.

Building	Geometry	Color	Scales With
Bank Tower	BoxGeometry stacked cubes	Deep blue #1A56DB	Savings proportion → height
Restaurants	BoxGeometry short + wide	Warm orange #F97316	Wants proportion → count
Apartments	BoxGeometry mid-height	Stone grey #9CA3AF	Needs proportion → count
Investment Tower	CylinderGeometry	Emerald #059669	Investment proportion → height
Parks / grass	PlaneGeometry + green material	Grass #22C55E	Overall health score → area
Citizens	CapsuleGeometry tiny	Pastel varied	Population = health score / 10

Animation — The useFrame Pattern

This is the most important Three.js pattern in the codebase. Never snap values — always lerp. The smooth transition is what makes the city feel alive.

```
// Inside BuildingDistrict.tsx
const targetHeight = useRef(1);
const meshRef = useRef<THREE.Mesh>(null);

// Read from Zustand
const savings = useGameStore(s => s.proportions.investments);
targetHeight.current = mapSavingsToBank(savings);

useFrame((_, delta) => {
  if (!meshRef.current) return;
  // Smooth interpolation — city always flows, never snaps
  meshRef.current.scale.y = THREE.MathUtils.lerp(
    meshRef.current.scale.y,
    targetHeight.current,
    delta * 2  // 2 = animation speed — adjust for feel
  );
});
```

Performance Rules for Demo Day

- Use InstancedMesh for building clusters — 50 buildings = 1 draw call, not 50
- Keep total polygon count under 50,000 — test with Stats from @react-three/drei
- FriendCity renders at 50% polygon count — it is in the distance, nobody counts faces
- Throttle useFrame if values haven't changed — only update GPU when Zustand state differs
- Test on the exact device you will demo on — not DevTools, not a faster MacBook
- If framerate drops below 45fps on demo device, reduce Citizens count first

SECTION 09

PWA Implementation Guide

Minimum viable PWA in under 1 hour

Is PWA Worth It in 36 Hours?

Yes — but only for installability and push notifications. Offline support is not worth the time. The PWA gives you two things judges notice: the app installs to the home screen like a real app, and push notifications arrive from the payment trigger. Both are visually impressive in the demo. The implementation takes under 1 hour using vite-plugin-pwa.

Setup in 4 Commands

```
npm install -D vite-plugin-pwa
```

Add to vite.config.ts:

```
import { VitePWA } from 'vite-plugin-pwa'
plugins: [react(), VitePWA({
  registerType: 'autoUpdate',
  manifest: {
    name: 'FinQuest', short_name: 'FinQuest',
    display: 'standalone', orientation: 'portrait',
    theme_color: '#111928', background_color: '#111928',
    start_url: '/',
    icons: [
      { src: '/icon-192.png', sizes: '192x192', type: 'image/png' },
      { src: '/icon-512.png', sizes: '512x512', type: 'image/png' }
    ]
  },
  workbox: { globPatterns: ['**/*.{js,css,html,ico,png,svg}'] }
})]
```

Push Notification Service Worker

Add a push event listener to your service worker. vite-plugin-pwa supports custom service worker code via the strategies option:

```
// sw.ts (custom service worker)
self.addEventListener('push', event => {
  const data = event.data?.json();
  event.waitUntil(self.registration.showNotification(data.title, {
    body: data.body,
    icon: '/icon-192.png',
    data: { url: data.url }  // deep link
  }));
});
self.addEventListener('notificationclick', event => {
  event.notification.close();
  event.waitUntil(clients.openWindow(event.notification.data.url));
```

```
});
```

PWA on iOS — Important Limitation

iOS requires Safari for PWA install. Push notifications on iOS PWAs require iOS 16.4+. Always demo the install and notification on a device you have tested. Do not assume it works on a device you have not tested. Confirm this by hour 14.

SECTION 10

Hosting & Deployment Guide

Where everything lives and how to deploy without breaking things

Service	Platform	Deploy Method	Notes
Frontend + API functions	Vercel	git push to main → auto-deploys in 60s	Connect GitHub repo on day 1 — every push auto-deploys
Langflow AI pipeline	Railway Docker	Deploy langflowai/langflow:latest image	Done once in hour 0-1 — Railway URL stored in Vercel env
Firebase Auth + DB	Firebase console	Firebase CLI or console setup	Done once in hour 0-1
3D assets (if any)	Vercel /public folder	Committed to Git repo	No separate CDN needed — Vercel serves /public efficiently
Demo video backup	YouTube unlisted or local file	Manual upload	Record at hour 30 as insurance

Environment Variables — Full List

Variable	Where Stored	Exposed To
VITE_FIREBASE_API_KEY	Vercel env + .env.local	Client (VITE_ prefix)
VITE_FIREBASE_PROJECT_ID	Vercel env + .env.local	Client
VITE_FIREBASE_APP_ID	Vercel env + .env.local	Client
VITE_VAPID_PUBLIC_KEY	Vercel env + .env.local	Client (needed for push subscribe)
FIREBASE_ADMIN_SDK	Vercel env (server only)	Server functions only
LANGFLOW_URL	Vercel env (server only)	Server functions only
LANGFLOW_API_KEY	Vercel env (server only)	Server functions only
LANGFLOW_ADVISOR_FLOW_ID	Vercel env (server only)	Server functions only
VAPID_PRIVATE_KEY	Vercel env (server only)	Server functions only
WEBHOOK_SECRET	Vercel env (server only)	Server + Shortcut payload

Backup Deployment

If Vercel fails during the hackathon, have Netlify as a backup. The same Vite build deploys identically. Keep netlify.toml in the repo as a file that is never needed but always available. It has saved hackathon teams before.

SECTION 11

Code Management & Collaboration

Git strategy for 4 people who want to stay unblocked

Monorepo — One Repo, Everything

Use a single monorepo. One GitHub repo, one Vercel project, one deployment. Vercel serverless functions live in /api. Frontend lives in /src. Langflow flow JSONs live in /langflow/flows. No monorepo tools needed — just a flat structure that everyone understands.

```
finquest/
  /src                ← React frontend (P1 + P2 primarily)
  /src/three          ← Three.js scene (P2 primarily)
  /api                ← Vercel serverless functions (P3 primarily)
  /langflow/flows     ← Exported Langflow flow JSONs
  /public             ← Static assets, PWA icons
  .env.local          ← Never committed (in .gitignore)
  .env.example        ← Committed, shows all required keys
```

Branching Strategy

Branch	Owner	Purpose
main	P4 (merge gatekeeper only)	Production — every push auto-deploys to Vercel
P1/ui	P1	All frontend screens, components, PWA
P2/3d	P2	All Three.js scene code
P3/ai	P3	Langflow flows, Vercel API functions, webhook
P4/firebase	P4	Firebase setup, Zustand stores, proportion engine, integration

Merge Rhythm

- P4 merges every 4 hours or when a major feature is complete
- Before merging, P4 pulls all branches, runs npm run build, confirms no TypeScript errors
- If there is a merge conflict, P4 resolves it — not the feature branch owner
- Never rebase during the hackathon — merge commits only, no history rewrites
- Every merge to main triggers a Vercel deploy — check it in 60 seconds, revert if broken

Shared Contracts — Define These in Hour 0

The four people building different systems need to agree on data shapes before they start. Define these in /src/types/index.ts and commit them in the first 30 minutes. Nobody waits for real data if the type contract exists.

```
// /src/types/index.ts — agree on these before building
type Category = 'need' | 'want' | 'treat' | 'investment';
type WeatherType = 'clear' | 'overcast' | 'rain' | 'storm';
interface Proportions { needs: number; wants: number; treats: number; investments:
number; }
interface CityState { bankHeight: number; restaurantCount: number; apartmentCount:
number; towerHeight: number; weather: WeatherType; population: number; }
interface AdvisorResponse { advisor_message: string; weather_effect: WeatherType;
building_changes: Partial<CityState>; tone: string; }
```

SECTION 12

Security, Testing & Reliability

What matters for a live demo — nothing more, nothing less

Security — Only What You Need

Concern	Action	Priority	What to Skip
VAPID private key exposed	Server-only env var — never in client bundle	CRITICAL	N/A
Anthropic/Langflow API key exposed	Server-only env var — proxy all AI calls through Vercel functions	CRITICAL	N/A
Webhook abuse during demo	Validate WEBHOOK_SECRET header — Shortcut sends it, function checks it	HIGH	Full rate limiting — not worth the time
Firebase open security rules	Leave open during demo — add rules post-hackathon	IGNORE	Full rules implementation
Input validation	Validate category enum on /api/webhook — reject anything malformed	MEDIUM	Zod schemas — overkill for hackathon
Prompt injection	Wrap user-derived text in Langflow prompts with data delimiters	MEDIUM	Full sanitisation pipeline

Testing Strategy

You have 36 hours. Test only what matters for the demo. Everything else is manual.

What to Test	How	When	Why
proportionEngine.ts	Vitest unit tests — 10 tests, 20 minutes	Hour 2-3	This is the math core — must be correct
cityMapper.ts	Manual console.log verification	Hour 4	Visual confirmation is faster than writing tests
Full demo loop	Manual on real devices	Hours 10, 14, 22, 28	The only test that matters for judges
Langflow flow	Langflow playground before integrating	Hour 5-6	Test in isolation before connecting to app
Web Push notification	Real iPhone test	Hour 10	Must work on the actual demo device
Multiplayer sync	Two physical devices, same room code	Hour 14	Two browser tabs is not the same as real devices

Smoke Tests — Run Before Submission

At hour 33, run every item on this list on the exact demo devices. Do not submit until all pass.

20. Sign in with Google on demo phone — works, city document created
21. Tap all 4 category buttons — proportions update, city buildings animate
22. Advisor message appears within 5 seconds of transaction
23. iOS Shortcut fires → notification arrives within 10 seconds → app opens to /log screen
24. Simulate Payment button triggers identical pipeline — works as backup
25. PWA installs from Safari on iPhone — opens in standalone mode
26. Two devices join same room code — both cities visible on both screens
27. Trigger world event — both cities' weather changes simultaneously
28. Langflow advisor called 5 times rapidly — no crash, no timeout
29. Vercel production URL loads in under 3 seconds on phone data (not WiFi)

SECTION 13

Demo & Judging Strategy

How to tell the story in 3 minutes and win

The 3-Minute Demo Script

Minute	What Happens	Who Does It	Judges See
0:00–0:30	Open the app installed on an iPhone. Show the city — slightly stormy, a few buildings. 'This is someone who just signed up with average spending habits.'	Speaker narrates, phone operator shows city	Beautiful isometric 3D city, immediately understandable
0:30–1:00	Make a real Apple Pay purchase on the phone. Wait. Notification arrives. 'Every time you pay, FinQuest asks one question — was that a Need, Want, Treat, or Investment?' Tap Investment.	Phone operator makes payment, taps Investment	The wow moment — real payment to app notification in real time
1:00–1:30	City reacts. Bank tower grows. Weather clears slightly. AI advisor appears: 'Great choice — your investment district is strengthening your city's foundations.' Show advisor message on screen.	Laptop operator shows city changing, advisor panel	AI feels personal and immediate. City changes are visual and dramatic.
1:30–2:00	'FinQuest is played with friends.' Switch to multiplayer view. Two cities side by side — one healthy, one struggling. Trigger world event. Both cities' weather darkens simultaneously.	Laptop operator switches to world view, triggers event	Social multiplayer, shared consequence — no other finance app does this
2:00–2:30	'We teach financial literacy through consequence, not instruction. No dollar amounts stored — ever. Just how you allocate. Fair for a student or a grad.'	Speaker narrates	Privacy angle, inclusivity, educational mission
2:30–3:00	Tech stack slide OR architecture diagram. 'Five integrated systems. React Three Fiber, Langflow RAG pipeline, Firebase real-time, PWA Web Push, iOS payment trigger.' Questions?	Speaker narrates diagram	Technical depth — judges reward systems thinking

Fallback Paths — Prepare These Before Demo Day

If This Fails	Do This Instead	Judges Notice?
iOS Shortcut doesn't fire	Tap the Simulate Payment button on screen — triggers identical pipeline	No — the notification still arrives on the phone
Langflow times out	Serve pre-cached advisor response from advisor_fallback.json	No — response appears the same way
Firebase sync lags	Pre-populate both cities before the demo starts — no live sync needed	Unlikely — but pre-populate as insurance
Web Push blocked by iOS	Show the notification arriving on a second device instead	Minor — show it somewhere
Three.js performance drops	Reduce Citizens count to 0 via URL param ?perf=low	No — city still looks good without citizens
WiFi fails entirely	Use phone hotspot — Vercel and Firebase both work on mobile data	No — plan for this in advance

What Judges Notice vs What They Don't

Judges Notice	Judges Don't Notice
The city changing visibly within 2 seconds of a tap	Your Firebase security rules being open
The notification arriving from a real phone payment	Whether your Langflow RAG has 10 or 1000 knowledge chunks
The AI advisor speaking in warm, personal language	The exact API response time as long as it feels fast
Two cities reacting simultaneously to a world event	Whether your TypeScript types are perfectly structured
The app being installed on a real phone home screen	Your folder structure and code organisation
You explaining the privacy model clearly and confidently	Whether you used a monorepo or multi-repo

SECTION 14

Build Checklists

Every task, in order, with no ambiguity

Build-This-First Checklist

If every item here is checked, you have a demoable, podium-contending product.

- GitHub repo created, Vite + React + TypeScript scaffolded, pushed to main
- Vercel connected to GitHub — auto-deploys on push confirmed
- Firebase project created: Auth, Realtime DB, Firestore enabled
- Railway account created, Langflow Docker image deployed, UI accessible
- Shared type contracts defined in /src/types/index.ts — all 4 members agree
- Google sign-in works on production Vercel URL (not localhost)
- New user: /cities/{cityId} and /worlds/{worldId} created in Firestore on first sign-in
- proportionEngine.ts written, manually tested with console.log
- Transaction logging screen: 4 buttons, merchant field, submit handler
- Submitting a transaction updates Zustand proportion state
- cityMapper.ts written: 5 mapping functions, manually verified
- Three.js canvas renders isometric scene with 5 placeholder buildings
- Building heights change smoothly when proportion state changes
- Weather system changes skybox colour based on health score
- Langflow Advisor Flow built, tested in playground, export flow ID
- /api/advisor Vercel function proxies to Langflow, returns JSON
- Advisor message displays in UI after transaction logged
- VAPID keys generated, stored in Vercel env vars
- /api/webhook Vercel function receives POST, validates secret, writes Firebase
- sendPushNotification function written, tested with manual trigger
- iOS Shortcut built, POSTs to production /api/webhook URL
- Full chain tested on real iPhone: Shortcut → push notification → app opens pre-filled
- Simulate Payment fallback button built and working
- PWA installable on iPhone Safari
- App installs to home screen and opens in standalone mode

Build-This-Later Checklist

- Firebase Realtime DB bidirectional sync with Zustand
- World/room join flow with 6-character room code
- FriendCity.tsx renders second city from room member Firebase data
- Shared world economic event via Langflow World Event Flow
- Streak indicator on city (flame icon, day count from Firestore)
- Rain particle system in Three.js
- Citizen capsule figures with count tied to health score

- Android Automate workflow
- Pattern Analyser Langflow flow
- Weekly settle animation sequence

SECTION 15

Risks, Tradeoffs & Simplifications

What could go wrong and exactly what to do about it

Risk	Likelihood	Impact	Exact Mitigation
iOS Shortcut fails live	Medium	Critical	Simulate Payment button — build this by hour 10, demo it at every rehearsal
Langflow cold start causes 8-second delay	High	High	Call /api/health 10 mins before demo to pre-warm Railway instance
Three.js framerate drops on demo laptop	Medium	High	Test on demo laptop by hour 14 — reduce Citizens and road detail if needed
Merge conflict chaos at hour 28	Medium	High	P4 is sole merge gatekeeper from hour 0 — no exceptions
Firebase Realtime DB sync lag in multiplayer demo	Low	High	Pre-populate both city states before judges watch — real sync then confirms
Push notification blocked on iPhone	Medium	Medium	Test exact demo device by hour 10 — iOS 16.4+ and Safari PWA install required
Claude API rate limit hit	Low	High	Pre-cache 5 advisor responses — serve from fallback.json on API error
Railway Langflow instance crashes	Low	Critical	LANGFLOW_BYPASS env var — when true, /api/advisor returns cached JSON immediately
Team member blocked and loses 4 hours	Medium	High	P4 owns integration — any blocker goes to P4 to resolve in under 30 minutes
Demo device runs out of battery	High	Critical	Every demo device plugged in 2 hours before demo. Backup device prepared.

Simplifications That Are Correct Decisions

- Proportion over magnitude — privacy-safe, income-fair, educationally sound, no sensitive data stored
- Procedural Three.js geometry over GLTF imports — faster, consistent aesthetic, zero asset pipeline
- Vercel serverless over Express server — zero DevOps, auto-scales, deploys in 60 seconds
- Firebase Realtime DB over WebSockets — managed, offline-persistent, no server code for sync
- iOS Shortcut over native Apple Wallet integration — Apple's platform restrictions make this the only web option
- 10 RAG text chunks over a full corpus — judges see the advisor output, not the knowledge base size
- Simulate Payment button over requiring Shortcut to work — demo reliability over technical purity

Final Principle — Repeat This at Every Team Sync

The 3-minute demo is the entire product from a judging perspective. Build what makes that demo work flawlessly. Polish what judges will see. Fake what judges will not see. Cut everything else. A flawless demo of a small product beats a broken demo of a big one every single time.

— End of FinQuest Hackathon Build Manual —

UNIHACK 2026 · 4 People · 36 Hours · Win the Room